CMP6202 Artificial Intelligence and Machine Learning

Predicting Stock Market Movements: A Sentiment-Driven Machine Learning Approach

Submitted By: Samikshya Dhamala

Student ID: 25123833

BSc (Hons) Computer Science with Artificial Intelligence

(Faculty of Computing, Engineering and the Built Environment)

Birmingham City University (BCU)

Sunway college Kathmandu, Nepal

Predicting Stock Market Movements: A Sentiment-Driven Machine Learning Approach

Abstract

Predicting short-term stock price movements is difficult due to market noise and non-linear behavior. This study uses supervised machine learning to predict one-day-ahead stock movements by combining historical price data, trading volume, and Reddit-based sentiment features. After removing the rare neutral class, the task was framed as a binary classification problem.

Exploratory data analysis and statistical tests guided limited feature engineering, with emphasis on price and volume features while retaining sentiment for potential non-linear effects. Random Forest, Gradient Boosting, and LightGBM models were trained using time-aware cross-validation. Hyperparameter tuning improved performance, with the tuned Random Forest achieving the best results (92.74% test accuracy and ROC-AUC of 0.9714).

SHAP analysis showed that price-based features dominate predictions, while sentiment contributes modestly at the instance level. Overall, the results indicate that combining sentiment with traditional financial indicators can improve predictive performance while preserving interpretability.

KEYWORDS: *Stock Price Movement Prediction; Machine Learning; Ensemble Models; Random Forest; LightGBM; Sentiment Analysis; Explainable Artificial Intelligence (XAI); SHAP; Financial Time Series*

Table of content

Abstract 2
Table of content 3
Introduction 4
Problem Statement 4
Literature review 4
Machine Learning Methodologies 4
Performance Analysis and Key Findings 5
Limitations and Challenges 5
Aims and Objectives 5
Methodology and Result 5
1.Dataset Description 5
Fig I: Dataset 6
Fig II: Data information before preprocessing 7
2. Data preprocessing and EDA(Exploratory Data Analysis) 7
Fig III: Data information after preprocessing 8
Fig IV: Target classification 9
Fig V: Probability heatmap of sentiment and target 10
Fig VI: Mann-Whitney U test output 12
Fig VII: Correlation heatmap of numeric features 12
Fig VIII:Time trend 13
Fig IX: KDE plot of numerical features 14
3.Train–Test Split 15
4.Feature Encoding and Feature Scaling 15
5.Model Selection and How it Works 16
6.Evalulation Matrix 18
7.Baseline Model 18
Fig X: Evaluation matrices before fine tuning 21
Fig XI: ROC curve of model before fine tunned 22
8.Hyperparameter Tuning 22
9.Tuned Model 23
Fig XII: Evaluation matrices after model fine tunned 24
10.Model Comparison 25
Fig XIII: Evaluation matrices comparison 25
Fig XIV: ROC curve of before and after model tunned 27
11.Model Interpretability (SHAP Analysis) 27
Fig XV: SHAP analysis 27
12.Real life prediction 28
Fig XVI: SHAP analysis on real life data 28
Reference list 29

Introduction

Predicting stock market movements is challenging due to market volatility and multiple influencing factors. Traditional models rely on historical prices (Agrawal and Mukherjee, 2025) but ignore external signals like news or social media. Sentiment-driven machine learning combines numerical data with public opinion from texts to improve prediction. This paper develops ML models to classify daily stock direction and multi-day forecasts, using SHAP to interpret feature contributions. By integrating market data, sentiment, and interpretable ML, the study provides a framework for more informed, transparent short-term trading decisions.

Problem Statement

This project aims to develop and evaluate different machine learning classification models to predict the next-day stock price movement (up or down) using historical market data and sentiment indicators.

Literature review

Machine Learning Methodologies

A wide range of machine learning algorithms has been explored for sentiment-driven stock prediction. Support Vector Machines (SVM) consistently demonstrate strong performance, with reported accuracies up to 98.48% when combined with sentiment analysis (Agrawal and Mukherjee, 2025, Koukaras, Nousi and Tjortjis, 2022). Long Short-Term Memory (LSTM) networks have been particularly effective in capturing sequential dependencies in stock price fluctuations (Agrawal and Mukherjee, 2025), while Random Forest models offer interpretability through feature importance analysis. Traditional algorithms such as Naive Bayes and K-Nearest Neighbors have also been tested to provide baseline comparisons (Mwamba, 2020). Comprehensive evaluations typically include accuracy, precision, recall, F1-score, and cross-validation techniques to ensure robustness.

Performance Analysis and Key Findings

Empirical evidence suggests that sentiment-driven approaches significantly improve predictive performance over price-only models. Agrawal et al. (2025) achieved 98.92% accuracy using BERT-based sentiment analysis, highlighting the advantage of deep learning models. Similarly, Nousi et al. (2021) reported an F1-score of 76.3% using VADER with SVM on Twitter data. Even traditional model stacking approaches, such as those employed by Bouktif et al. (2020), achieved competitive results with 60% accuracy, underscoring the importance of integrating sentiment features. Overall, investor sentiment and public emotions derived from textual data are significant predictors of stock market movements.Empirical evidence suggests that sentiment-driven approaches significantly improve predictive performance over

price-only models. Agrawal et al. (2025) achieved 98.92% accuracy using BERT-based sentiment analysis, highlighting the advantage of deep learning models. Similarly, Nousi et al. (2021) reported an F1-score of 76.3% using VADER with SVM on Twitter data. Even traditional model stacking approaches, such as those employed by Bouktif et al. (2020), achieved competitive results with 60% accuracy, underscoring the importance of integrating sentiment features. Overall, investor sentiment and public emotions derived from textual data are significant predictors of stock market movements.

Aims and Objectives

The main aim of this work is to develop machine learning models to predict stock price direction using historical market and sentiment data.
The objectives of this work are to preprocess and analyze the dataset, perform feature engineering, train and evaluate multiple machine learning models, compare their performance before and after hyperparameter tuning, and interpret the results as well.

Methodology and Result

1.Dataset Description

The dataset to predict stock price movement is downloaded from a kaggle "https://www.kaggle.com/datasets/rootpi3/stock-price-prediction-with-sentiment-analysis/data" and the dataset contains 14 columns and 12069 rows. The dataset used in this study integrates historical stock price data with Reddit-derived sentiment information to support stock price movement prediction. It is structured to allow exploration of the relationship between market sentiment and stock behavior, making it suitable for machine learning applications.

Adj Close	Close	High	Low	Open	Volume	Company	Date	Target	Score	Comments	Cleaned_Text	Sentiment	Sentiment_Score
8.532785416	10.1153574	10.13500023	9.85178566	9.859999657	658677600	Apple	2010-09-20	1	0	0	stock market gar	Positive	0.99989474
8.712499619	8.712499619	8.897000313	8.686499596	8.81649971	84050000	Amazon	2010-12-13	1	7	5	hussman warnin	Neutral	0.5534017
8.778499603	8.778499603	8.949999809	8.679499626	8.68599987	116210000	Amazon	2010-12-15	1	8	0	awful time invest	Negative	0.98214895
8.887499809	8.887499809	8.987500191	8.727999687	8.843000412	93130000	Amazon	2011-02-24	1	5	1	amazon prime sl	Negative	0.97737736
11.44633389	13.56928635	13.60285664	13.28214264	13.32142925	467832400	Apple	2011-09-12	-1	5	16	personally im far	Negative	0.99398685
12.14617348	14.39892864	14.4278574	13.97500038	14.28071404	812876400	Apple	2011-09-26	-1	8	18	wondering inves	Positive	0.8621679
11.22218895	13.30357075	13.63571358	12.65142918	13.37749958	1233677200	Apple	2011-10-04	-1	8	18	sprint debt worry	Positive	0.9998833
12.72068882	15.07999992	15.17178631	14.85678577	15.06285667	881602400	Apple	2011-10-18	-1	28	58	first miss since 2	Negative	0.9999378
12.72068882	15.07999992	15.17178631	14.85678577	15.06285667	881602400	Apple	2011-10-18	-1	10	5	apple miss big e	Negative	0.99999666
11.35750008	11.35750008	11.95049953	11.29450035	11.92949963	280252000	Amazon	2011-10-25	1	16	12	look like nuclear	Positive	0.99919575

Fig I: Dataset

Data Structure:

The dataset contains the following columns:

- Date: The date of the recorded data (YYYY-MM-DD format).
- Company: The company associated with the stock data (e.g., Tesla, Apple,Amazon).
- Open, High, Low, Close: Daily stock price metrics representing the opening, highest, lowest, and closing prices of the stock.
- Volume: The number of shares traded during the day.
- Target: The label indicating the stock price movement for the next day:

    - 1 → Price increased
    - -1 → Price decreased
    - 0 → No significant change
- Score: Popularity score of the Reddit post, based on upvotes.
- Comments: Number of comments on the Reddit post.
- Cleaned_Text: Preprocessed Reddit post text, cleaned of special characters, stopwords, and irrelevant content.
- Sentiment: Sentiment label of the Reddit post: Positive (Optimistic), Neutral, or Negative (Pessimistic).
- Sentiment_Score: Numerical representation of sentiment strength derived from different analysis.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12069 entries, 0 to 12068
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Adj Close        12069 non-null  float64
 1   Close            12069 non-null  float64
 2   High             12069 non-null  float64
 3   Low              12069 non-null  float64
 4   Open             12069 non-null  float64
 5   Volume           12069 non-null  int64
 6   Company          12069 non-null  object
 7   Date             12069 non-null  object
 8   Target           12069 non-null  int64
 9   Score            12069 non-null  int64
 10  Comments         12069 non-null  int64
 11  Cleaned_Text     11915 non-null  object
 12  Sentiment        12069 non-null  object
 13  Sentiment_Score  12069 non-null  float64
dtypes: float64(6), int64(4), object(4)
memory usage: 1.3+ MB
```

Fig II: Data information before preprocessing

Since the target is known and labeled, this problem falls under supervised learning. Specifically, it is a classification task, where the goal is to assign each day's data to one of three classes (increase, decrease, no change) based on features.

2. Data preprocessing and EDA(Exploratory Data Analysis)

A. **Date Handling:** The Date column was converted to a datetime format and the dataset was sorted chronologically to preserve temporal ordering. This ensures correct next-day target generation and prevents data leakage during model training and evaluation.

B. **Missing Values:** Missing values in the Cleaned_Text column were identified (154 entries) and subsequently removed using dropna(), resulting in a dataset of 11915 entries.

C. **Duplicate Rows**: A check for duplicate rows confirmed that there were none in the dataset.

```
<class 'pandas.core.frame.DataFrame'>
Index: 11915 entries, 0 to 12068
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Adj Close        11915 non-null  float64
 1   Close            11915 non-null  float64
 2   High             11915 non-null  float64
 3   Low              11915 non-null  float64
 4   Open             11915 non-null  float64
 5   Volume           11915 non-null  int64
 6   Company          11915 non-null  object
 7   Date             11915 non-null  datetime64[ns]
 8   Target           11915 non-null  int64
 9   Score            11915 non-null  int64
 10  Comments         11915 non-null  int64
 11  Cleaned_Text     11915 non-null  object
 12  Sentiment        11915 non-null  object
 13  Sentiment_Score  11915 non-null  float64
dtypes: datetime64[ns](1), float64(6), int64(4), object(3)
memory usage: 1.4+ MB
```

Fig III: Data information after preprocessing

D. **Feature Engineering**: Year, Month, and DayOfWeek were extracted from the Date column to represent basic temporal information.
Minimal preprocessing and minimal feature engineering is common here because:

1. The dataset is information-rich, consisting of stock prices, trading volume, and sentiment scores.
2. Excessive feature engineering was avoided to reduce noise and prevent overfitting in time-ordered financial data.
3. Creating numerous technical indicators or lag features can introduce spurious patterns in short-term prediction tasks.
4. Ensemble models such as Random Forest, Gradient Boosting, and LightGBM can automatically capture non-linear relationships and feature interactions.

E. **Target Class Filtering:** The Target variable, which classifies stock movement, initially contained a rare neutral class (Target = 0, only 2 instances). The neutral class exhibits significantly lower return volatility, indicating price separation rather than a distinct market regime. So, neutral class should be removed because:
   - Extremely rare class: 2 samples out of, say, 10,000 → almost nothing to train on
   - Statistical instability: Any test (t-test, chi-square, etc.) would be unreliable
   - Model confusion: Most ML classifiers will essentially ignore this class or overfit to it

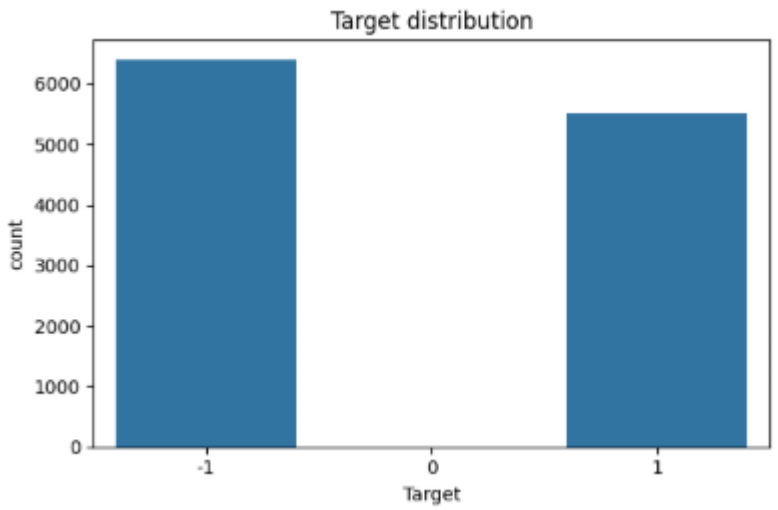


Fig IV: Target classification

To maintain statistical power and enable stable two-group comparisons, this class was excluded from subsequent statistical testing. So,this class was removed from the dataset to ensure statistical stability and focus on clear upward (1) or downward (-1) movements for modeling.

F. **Target Distribution:** After removing the rare neutral class, the dataset contained 5513 upward movements (Target = 1) and 6400 downward movements (Target = -1), showing a slight imbalance towards downward movements. Since the imbalance was minor (~7%), resampling techniques such as SMOTE(Synthetic Minority Over-sampling Technique) were deemed unnecessary for model training. Class distribution was preserved to maintain the natural characteristics of market movements.

G. **Sentiment effects on target**

Fig V: Probability heatmap of sentiment and target

A conditional probability analysis was performed to examine the relationship between Sentiment and Target stock movements. Negative, neutral, and positive sentiments all exhibit probabilities close to 50% for upward or downward movements, indicating that sentiment alone is not strongly predictive of stock movement. However, sentiment features may still provide marginal predictive power when combined with price-based features.

H. **Categorical Features vs. Target (Chi-square Test)**

The Chi-Square (χ^2) Test is a statistical method which compares the observed frequency of occurrences in each category with the frequencies that would be expected if there were no association between the variables.

The test statistic is calculated as:

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

Where:

- Oi = Observed frequency in category i
- Ei = Expected frequency in category i under the null hypothesis

In my project:

	feature	chi2	p_value	cramers_v	n	p_fdr_bh
0	Company	270.539133	1.791322e-59	0.150697	11913	3.582645e-59
1	Sentiment	2.983703	2.249557e-01	0.015826	11913	2.249557e-01

- Company: The Chi-square test revealed a statistically significant association between Company and Target (very low p-value), suggesting different companies might exhibit different target movement patterns. However, the effect size (Cramer's V = 0.15) was weak.
- Sentiment: The Sentiment feature (Positive, Neutral, Negative) was not found to be statistically significant with respect to the Target (p-value = 0.225), indicating no strong association.

I. **Numerical Features vs. Target (Mann-Whitney U Test)**

The Mann-Whitney U Test is a non-parametric test used to compare differences between two independent groups when the dependent variable is either ordinal or continuous but does not follow a normal distribution.

The test works by ranking all observations from both groups together, then comparing the sum of ranks between groups. The U statistic is calculated as:

$$U = n_1 n_2 + \frac{n_1(n_1+1)}{2} - R_1$$

Where:

- n1 and n2 = sample sizes of the two groups
- R1 = sum of ranks for the first group

A small p-value (typically < 0.05) indicates a significant difference between the two groups.

In my project:

	feature	p_mwu	cliffs_delta	cohens_d	mean_target1	\
7	Target	0.000000e+00	1.000000	NaN	1.000000e+00	
2	High	2.754879e-203	-0.322761	-0.577998	1.234291e+02	
1	Close	9.299264e-202	-0.321533	-0.572667	1.216617e+02	
0	Adj Close	7.011365e-201	-0.320826	-0.574023	1.208824e+02	
4	Open	2.891920e-199	-0.319520	-0.569445	1.217829e+02	
3	Low	6.267833e-197	-0.317621	-0.562899	1.199366e+02	
5	Volume	4.902357e-33	-0.127028	-0.098751	1.125133e+08	
11	Year	6.896139e-07	0.052153	0.097278	2.020991e+03	
6	Date	1.742353e-06	0.050725	0.094404	1.624513e+18	
9	Comments	3.336833e-02	0.022571	0.054562	4.823630e+02	
13	DayOfWeek	4.065640e-02	-0.021274	-0.037859	1.990931e+00	
12	Month	2.049831e-01	-0.013399	-0.023561	6.378560e+00	
8	Score	4.828310e-01	-0.007437	-0.044633	9.265030e+02	
10	Sentiment_Score	8.149593e-01	-0.002483	-0.000648	9.591114e-01	

	mean_target-1	n1	n-1	p_fdr_bh
7	-1.000000e+00	5513	6400	0.000000e+00
2	1.695218e+02	5513	6400	1.928415e-202
1	1.663349e+02	5513	6400	4.339656e-201
0	1.658208e+02	5513	6400	2.453978e-200
4	1.662243e+02	5513	6400	8.097376e-199
3	1.628454e+02	5513	6400	1.462494e-196
5	1.225595e+08	5513	6400	9.804715e-33
11	2.020729e+03	5513	6400	1.206824e-06
6	1.616485e+18	5513	6400	2.710326e-06
9	3.636609e+02	5513	6400	4.671567e-02
13	2.043281e+00	5513	6400	5.174451e-02
12	6.459062e+00	5513	6400	2.391469e-01
8	1.099707e+03	5513	6400	5.199718e-01
10	9.591771e-01	5513	6400	8.149593e-01

Fig VI: Mann-Whitney U test output

Highly Significant Features: Adj Close, Close, High, Low, Open, Volume, Year, Date, Comments, and DayOfWeek were all found to be extremely significant predictors of the Target (very low p-values after FDR correction) with moderate to large effect sizes. This implies their distributions differ significantly between days of positive vs. negative stock movement.

Insignificant Features: Month, Score, and Sentiment_Score were not statistically significant (high p-values), meaning their distributions do not reliably differ between positive and negative target days.

J. **Correlation Heatmap**

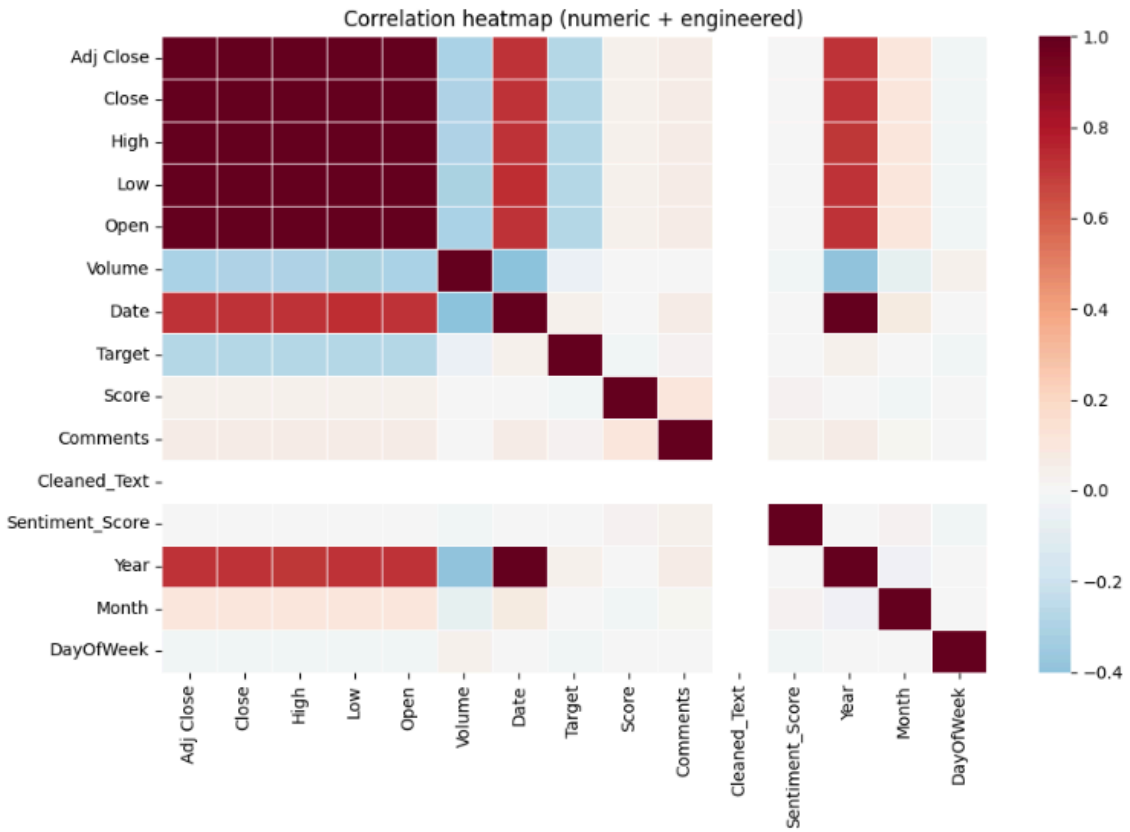


Fig VII: Correlation heatmap of numeric features

Strongly Correlated Price Features: As expected, Adj Close, Close, High, Low, and Open show very strong positive correlations with each other (values close to 1). This is typical for stock price data, indicating that these metrics move together almost in unison.

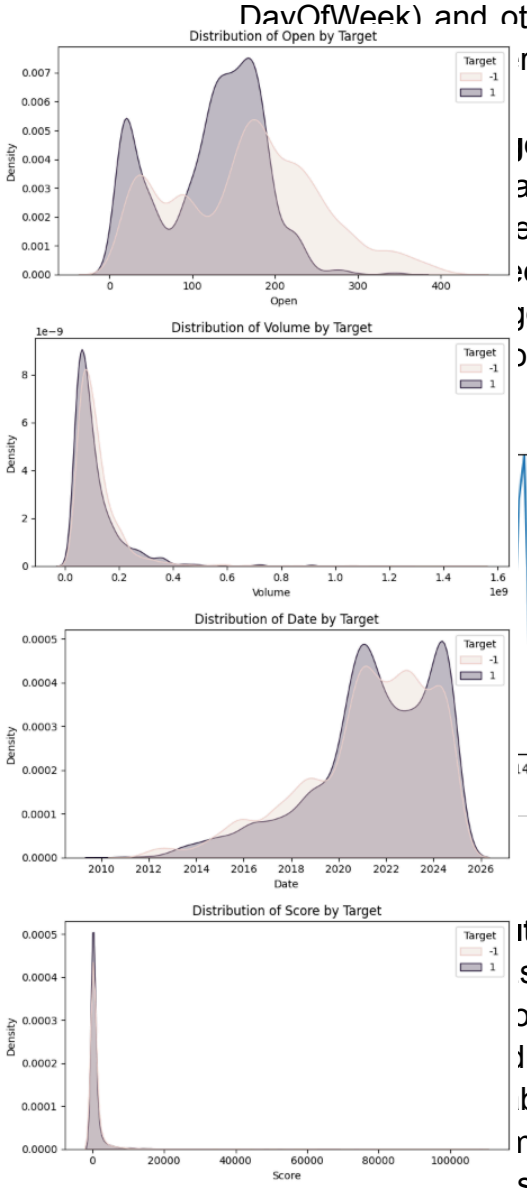
Volume Correlation: Volume shows some correlation with the Target. The Mann-Whitney U test indicated that Volume is statistically significant (p_fdr_bh is very low) with a moderate cliffs_delta effect size, suggesting trading activity has a meaningful difference across target classes.

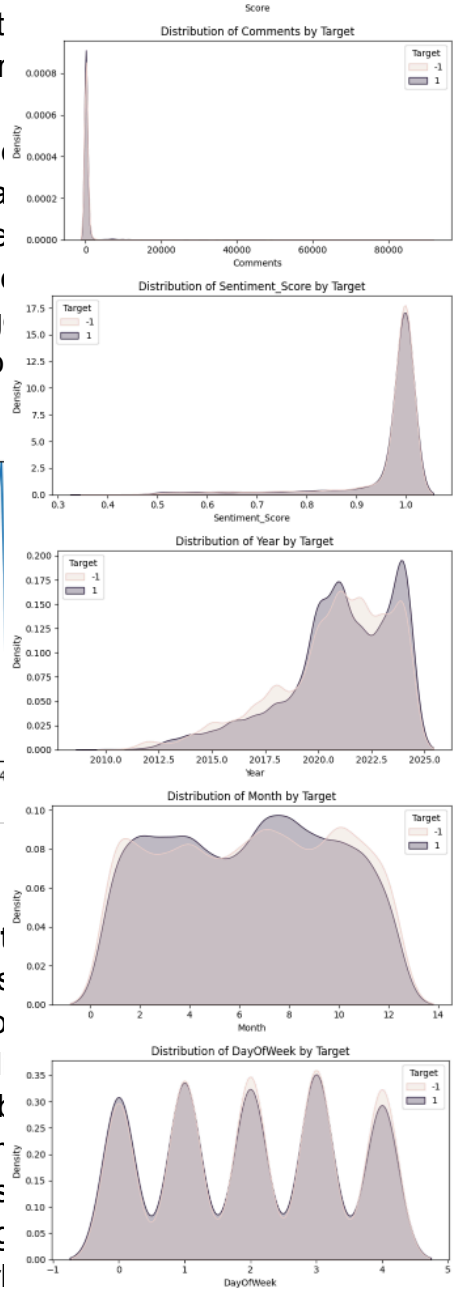
Significant Numerical Features (Mann-Whitney U Test): The statistical tests on numerical features (num_tests DataFrame) highlight that Adj Close, Close, High, Low, Open, Volume, Year, Date, Comments, and DayOfWeek are all highly significant predictors of the Target (extremely low p-values after FDR correction). This means there are statistically significant differences in these features between when the Target is 1 versus -1. So, these features display noticeable, non-zero correlation values with Target on the heatmap.

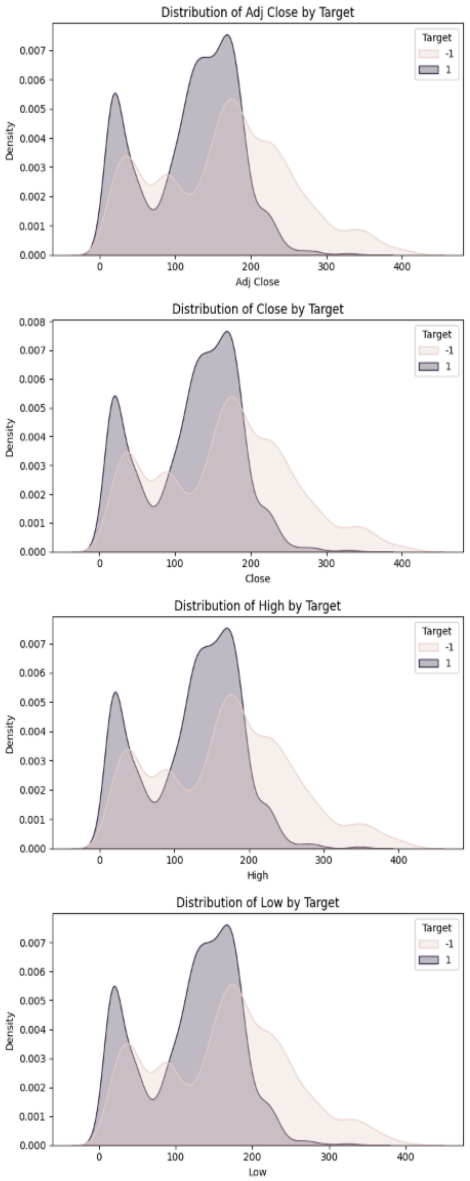
Weak or Insignificant Numerical Features: Features such as Month, Score, and Sentiment_Score showed higher p_fdr_bh values (not significant) in the Mann-Whitney U test, and very small cliffs_delta values. On the heatmap, I

observe that their correlation with the Target and potentially many other features is very close to zero, indicating minimal linear relationship and limited predictive power based on their raw values.

Engineered Features: The engineered date fe
DayOfWeek) and ot







Mann-Whitney U tes ant differences in their distributions for Targ s insignificant features showed largely overl

Fig IX: KDE plot of numerical features

Significant Numerical Features (Adj Close, Close, High, Low, Open, Volume, Year, Date, Comments, DayOfWeek):
Mann-Whitney U tests show very low p-values and moderate to large effect sizes, indicating these features differ significantly between Target = 1 and Target = -1. Box and KDE plots reveal clear differences in price levels, volumes, temporal patterns, comment activity, and day-of-week effects between upward and downward movement days.
Insignificant Features (Month, Score, Sentiment_Score):
High p-values and small effect sizes suggest no statistical difference between targets. Visualizations show overlapping distributions, confirming these features do not distinguish stock movements.

M. Final feature selection

Feature selection was guided by exploratory analysis using the Mann–Whitney U test (numerical) and Chi-square test (categorical). Price and volume variables (Adj Close, Close, High, Low, Open, Volume) and the Company feature were retained due to strong statistical significance with the Target. Sentiment was kept despite being non-significant to allow models to capture potential non-linear effects. Features like Score, Sentiment_Score, Month, and DayOfWeek were excluded due to weak relevance, while Year was dropped to avoid overfitting. Date, Comments, and Cleaned_Text were retained to preserve temporal structure and support textual/contextual learning.

3.Train–Test Split

The dataset was split into features (X) and target (y), excluding Score, Sentiment_Score, Year, Month, and DayOfWeek based on feature selection. The remaining features, including price, volume, company, Date, Comments, Cleaned_Text, and Sentiment, were used for modeling. Data was then divided into training (80%) and testing (20%) sets to train the model and evaluate its performance on unseen data.

4.Feature Encoding and Feature Scaling

To prepare the data for machine learning models, numerical features (Adj Close, Close, High, Low, Open, Volume) were first scaled using standardization after filling missing values with the mean. Scaling ensures that all numerical features are on a similar scale, preventing features with larger values from dominating the model.

Categorical features (Company, Sentiment) were one-hot encoded after imputing missing values with a placeholder. One-hot encoding converts categorical data into a numerical format that machine learning models can process, representing each category as a separate binary column.

Combining these steps into a single preprocessing pipeline ensures consistent and efficient transformation of all features before model training.

5.Model Selection and How it Works

For predicting stock movements, Random Forest, Gradient Boosting, and LightGBM were selected due to their strong performance on tabular financial data. These tree-based ensemble models effectively capture non-linear relationships and feature interactions, handle both numerical and categorical features without heavy scaling, and provide feature importance insights to understand predictive drivers. They are well-suited for binary classification tasks, offering reliable probability estimates for evaluation. Neural networks or other deep learning models were not used because the dataset contains only 12.1K rows, which is relatively small for large models and could lead to overfitting. Initial experiments and hyperparameter tuning confirmed

that these ensemble models achieve high predictive performance, making them appropriate for this task.
Deep learning models (e.g., neural networks) were not used because the dataset contains only 12.1K rows, which is small for training large models. Deep learning requires much larger datasets to generalize effectively and avoid overfitting, whereas ensemble tree-based models can achieve strong performance with smaller, structured datasets while remaining faster and easier to train.

How the Random Forest Classifier works step by step

1. **Bootstrap Sampling**
   During training, the Random Forest algorithm creates multiple bootstrap samples from the training dataset by sampling with replacement. Each bootstrap sample contains a slightly different subset of the original data.
2. **Decision Tree Construction**
   For each bootstrap sample, a decision tree is grown.
   - At each node, only a random subset of features is considered for splitting.
   - The split is chosen to maximize class separation using impurity measures such as Gini impurity.
   - This randomness ensures that trees are decorrelated and capture diverse patterns.
3. **Learning from Financial Features**
   In this study, trees learn decision rules based on:
   - Price structure (Open, High, Low, Close, Adjusted Close)
   - Market activity (Volume)
   - Categorical effects (Company, Sentiment)
4. Price-based features dominate most splits, consistent with the statistical analysis, while sentiment and company features act as secondary modifiers.

5. **Ensemble Prediction**
   During inference, each tree outputs a class probability (up or down).
   The final prediction is obtained by averaging probabilities across all trees, followed by thresholding to determine direction.

How the Gradient Boosting Classifier works step by step

1. **Initialization**
   The model starts with a simple baseline predictor, typically predicting the average class probability across the training data.
2. **Residual-Based Learning**
   - The first decision tree is trained to predict the residual errors from the baseline model.

    - Subsequent trees are trained on the residuals of the previous ensemble.
    - Each tree learns patterns that were previously misclassified.
3. **Gradient Optimization**
   Gradient Boosting minimizes a loss function (e.g., log-loss for classification) using gradient descent in function space.
    - Each tree moves the model in the direction that reduces classification error.
    - A learning rate controls how much each tree contributes, preventing overfitting.
4. **Capturing Subtle Market Signals**
   In my dataset, Gradient Boosting learns:
    - Fine-grained price movements
    - Interactions between sentiment and price
    - Temporal inconsistencies that single trees cannot capture
5. **Final Prediction**
   The outputs of all trees are combined as a weighted sum to produce a final probability estimate for upward movement.

How the LightGBM Classifier works step by step

1. **Histogram-Based Splitting**
   Continuous features are discretized into bins, reducing computation and memory usage.
   This is particularly effective for numerical stock price data.
2. **Leaf-Wise Tree Growth**
   Unlike traditional level-wise growth, LightGBM grows trees leaf-wise, expanding the leaf that leads to the largest reduction in loss.
    - This enables more complex decision boundaries
    - Controlled using parameters like max_depth and num_leaves to prevent overfitting
3. **Efficient Gradient Optimization**
   Similar to Gradient Boosting, LightGBM uses gradient-based optimization but does so more efficiently, allowing deeper trees without excessive computational cost.
4. **Handling Feature Interactions**
   LightGBM effectively captures:
    - Non-linear price relationships
    - Volume-driven volatility effects
    - Conditional influence of sentiment under specific price regimes
5. **Final Prediction**
   The model aggregates contributions from all boosted trees to produce class probabilities, which are then converted into directional predictions.

6.Evalulation Matrix

Classification models are evaluated using metrics derived from the confusion matrix, which includes:

- TP (True Positive): Correctly predicted positive cases
- TN (True Negative): Correctly predicted negative cases
- FP (False Positive): Incorrectly predicted positive cases
- FN (False Negative): Incorrectly predicted negative cases

From these, key metrics are calculated:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F1\text{-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Accuracy: Ratio of correctly predicted instances to total instances.
- Precision: Proportion of correctly predicted positive cases out of all predicted positives.
- Recall (Sensitivity): Proportion of correctly predicted positive cases out of all actual positives.
- F1-Score: Harmonic mean of precision and recall, useful for imbalanced datasets.
- Confusion Matrix: Table showing true positives, true negatives, false positives, and false negatives.
- ROC-AUC: Measures the model's ability to distinguish between classes.

7.Baseline Model

1. CV Accuracy (Mean Cross-Validation Accuracy):

This represents the average accuracy across the folds of time-series cross-validation. It estimates how well the model generalizes to unseen data within the training period and indicates model stability.

- RandomForest: 0.8144 (81.44%)
- GradientBoosting: 0.7437 (74.37%)
- LightGBM: 0.7936 (79.36%)

RandomForest shows the highest and most consistent cross-validation performance, closely followed by LightGBM. GradientBoosting performs significantly lower, suggesting less stability and weaker generalization during training.

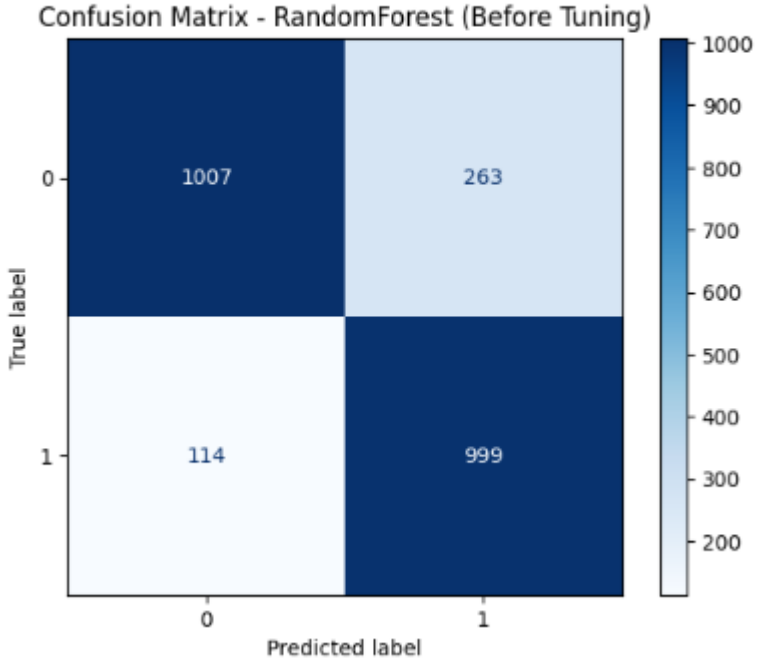
2. Test Accuracy:

This measures the model's performance on a completely unseen test set, indicating how well it generalizes to new data.

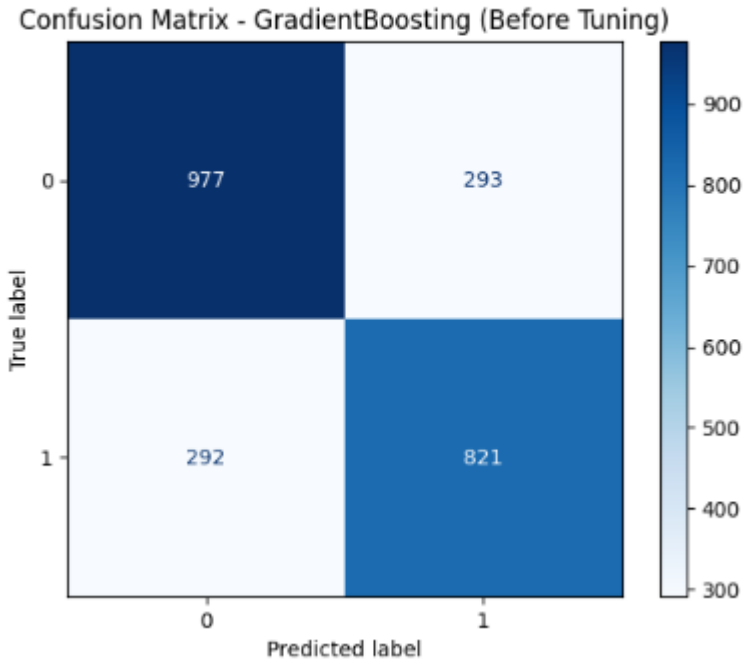
- RandomForest: 0.8418 (84.18%)
- GradientBoosting: 0.7545 (75.45%)
- LightGBM: 0.8175 (81.75%)

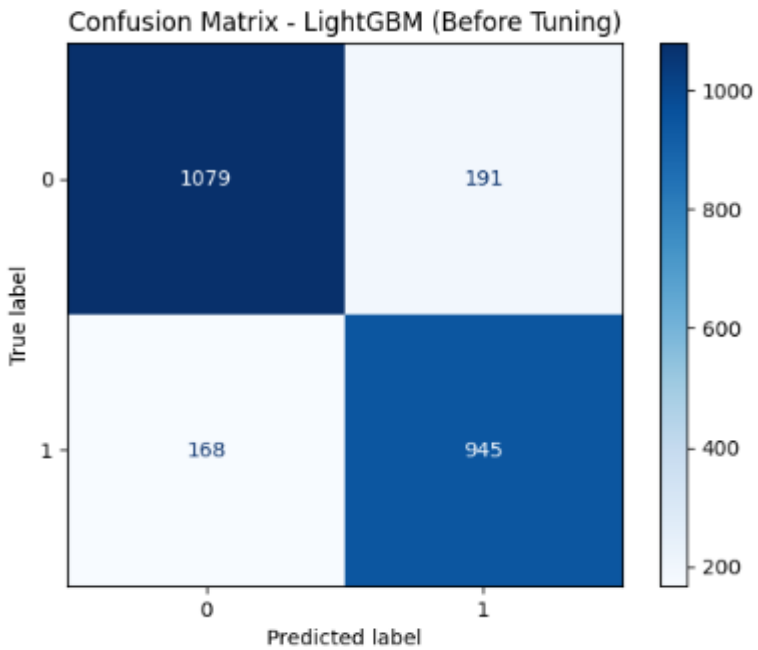
RandomForest leads in test accuracy, confirming its strong generalization. LightGBM performs well, while GradientBoosting remains notably weaker.

```
--- RandomForest ---
Mean CV Accuracy: 0.8144
Test Accuracy: 0.8417960553923626
```




```
--- GradientBoosting ---
Mean CV Accuracy: 0.7437
Test Accuracy: 0.7545111204364247
```





Model	CV Accuracy	Test Accuracy	ROC AUC	Precision (weighted)	Recall (weighted)	F1-Score (weighted)	Accuracy
RandomForest	0.814358	0.841796	0.928970	0.848468	0.841796	0.841829	0.841796
GradientBoosting	0.743703	0.754511	0.841018	0.754525	0.754511	0.754518	0.754511
LightGBM	0.820781	0.849350	0.931688	0.849672	0.849350	0.849432	0.849350

Fig X: Evaluation matrices before fine tuning

3. ROC AUC (Area Under the Receiver Operating Characteristic Curve):
ROC AUC evaluates the model's ability to distinguish between positive and negative classes across all classification thresholds.

- RandomForest: 0.9238
- GradientBoosting: 0.8407
- LightGBM: 0.9304

LightGBM achieves the highest ROC AUC, demonstrating the best overall discriminatory power among the untuned models. RandomForest is very close, while GradientBoosting lags behind.

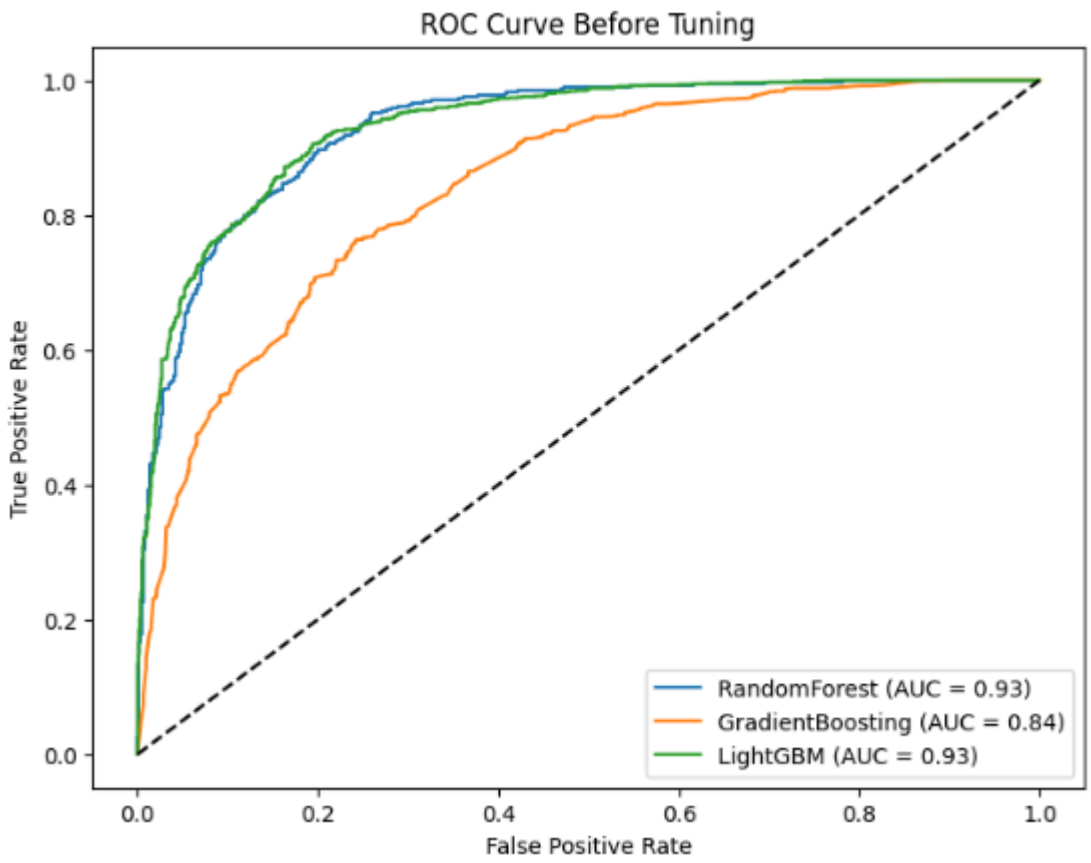


Fig XI: ROC curve of model before fine tunned

4. Precision, Recall, and F1-Score (Weighted):
   - Precision (weighted): Measures the model's ability to avoid false positives, averaged across classes.
   - Recall (weighted): Measures the ability to capture all positive samples, averaged across classes.
   - F1-Score (weighted): Harmonic mean of precision and recall, balancing both metrics.

8.Hyperparameter Tuning

To optimize the performance of our machine learning models, we performed hyperparameter tuning using GridSearchCV, which systematically evaluates combinations of parameters to find the best-performing configuration.

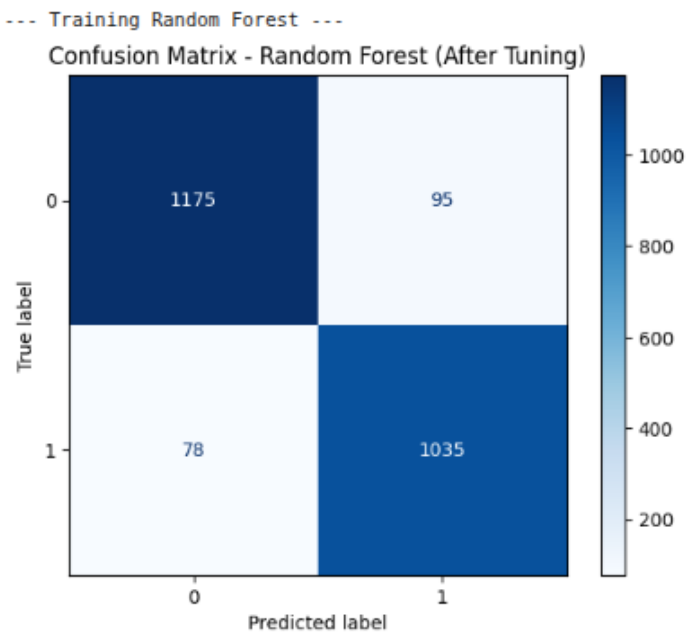
1. Random Forest
   - n_estimators: Number of trees in the forest. Tested values: 100, 200. Increasing the number of trees can improve performance but also increases computation time.
   - max_depth: Maximum depth of each tree. Tested values: 5, 10, 15. Controlling tree depth helps prevent overfitting by limiting the complexity of each tree.
2. Gradient Boosting

- n_estimators: Number of boosting stages. Tested values: 100, 200. More estimators can improve learning but may also increase overfitting risk.
- learning_rate: Step size shrinkage used in updates. Tested values: 0.01, 0.1, 0.5. Lower values require more trees but can lead to better generalization.

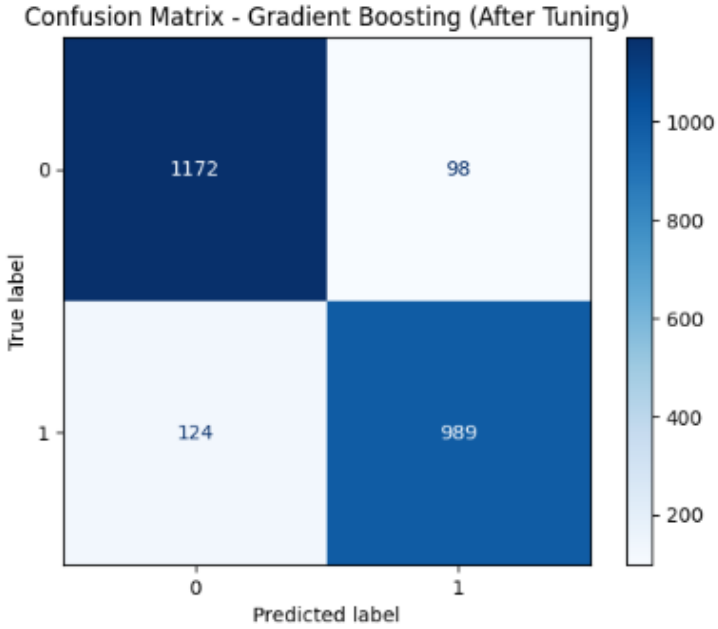
3. LightGBM
   - n_estimators: Number of boosting iterations. Tested values: 100, 200.
   - learning_rate: Step size shrinkage. Tested values: 0.01, 0.1, 0.3.
   - max_depth: Maximum depth of the trees. Tested values: 3, 5, 7. Limits overfitting by controlling tree complexity.
   - num_leaves: Maximum number of leaves per tree. Tested values: 31, 50. Larger values allow more complex tree structures, potentially improving performance but increasing overfitting risk.

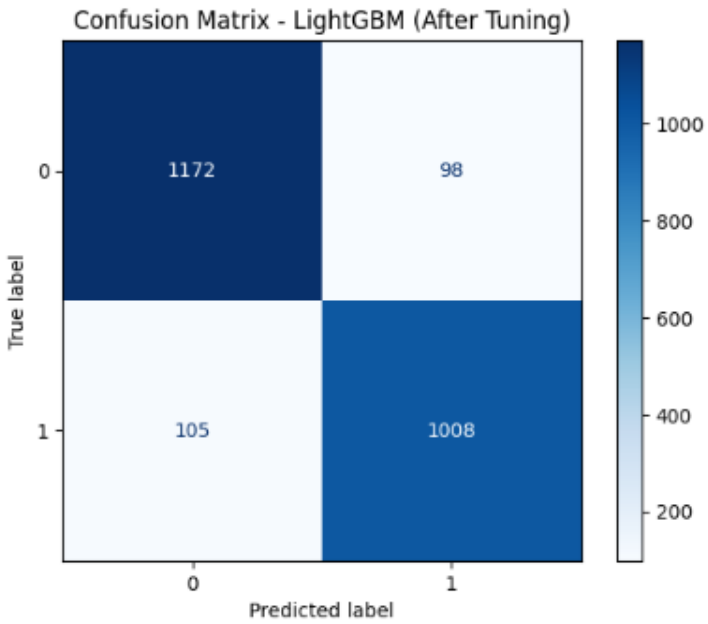
GridSearchCV evaluates all possible combinations of these parameters using cross-validation and selects the configuration that maximizes model performance (e.g., accuracy or ROC AUC). By tuning these hyperparameters, we aim to enhance the predictive power of the models while minimizing overfitting and ensuring better generalization on unseen test data.

9.Tuned Model



```
--- Training Gradient Boosting ---
```






```
                   CV Accuracy  Test Accuracy   ROC AUC  Precision (weighted)  \
Model
Random Forest         0.865113       0.927402  0.971412              0.927558
Gradient Boosting     0.855164       0.906840  0.953518              0.906900
LightGBM              0.865869       0.914813  0.968364              0.914795

                   Recall (weighted)  F1-Score (weighted)  Accuracy
Model
Random Forest               0.927402             0.927433  0.927402
Gradient Boosting           0.906840             0.906762  0.906840
LightGBM                    0.914813             0.914796  0.914813
```

Fig XII: Evaluation matrices after model fine tunned

10.Model Comparison

After performing hyperparameter tuning using GridSearchCV, we evaluated the models again to assess the improvement over their baseline performance. The comparison of metrics before and after tuning demonstrates the effectiveness of optimization in enhancing predictive performance.

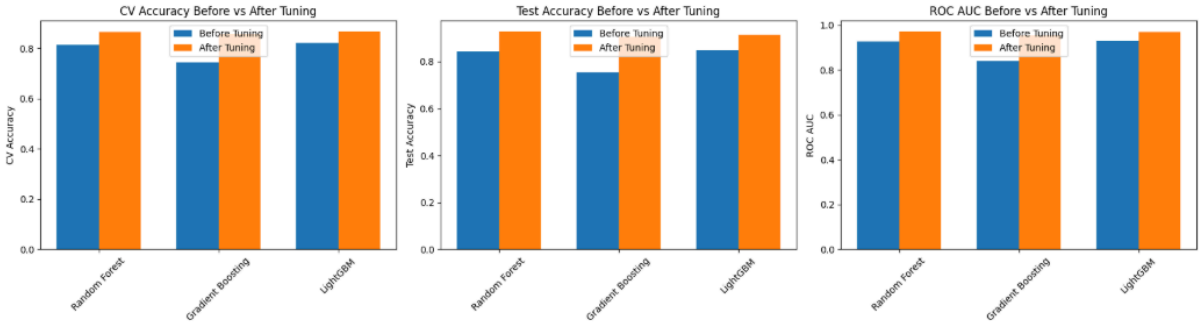


Fig XIII: Evaluation matrices comparison

1. Random Forest Classifier
   - CV Accuracy: 0.8144 → 0.8651
   - Test Accuracy: 0.8418 → 0.9274
   - ROC AUC: 0.9238 → 0.9714
   - Precision (weighted): 0.8427 → 0.9276
   - Recall (weighted): 0.8418 → 0.9274
   - F1-Score (weighted): 0.8419 → 0.9274

Random Forest exhibited substantial improvements across all metrics. This indicates that the chosen hyperparameters (e.g., n_estimators=200, max_depth=10) effectively optimized model performance, making it highly accurate and balanced in predicting both classes.

2. Gradient Boosting Classifier
   - CV Accuracy: 0.7437 → 0.8552
   - Test Accuracy: 0.7545 → 0.9068
   - ROC AUC: 0.8407 → 0.9535
   - Precision (weighted): 0.7547 → 0.9069
   - Recall (weighted): 0.7545 → 0.9068
   - F1-Score (weighted): 0.7544 → 0.9068

Gradient Boosting saw the most dramatic improvement after tuning.This demonstrates that the default hyperparameters were suboptimal and tuning (e.g., n_estimators=200, learning_rate=0.01) unlocked its full potential.

3. LightGBM Classifier
   - CV Accuracy: 0.7936 → 0.8659
   - Test Accuracy: 0.8175 → 0.9148
   - ROC AUC: 0.9304 → 0.9684
   - Precision (weighted): 0.8188 → 0.9148
   - Recall (weighted): 0.8175 → 0.9148
   - F1-Score (weighted): 0.8177 → 0.9148

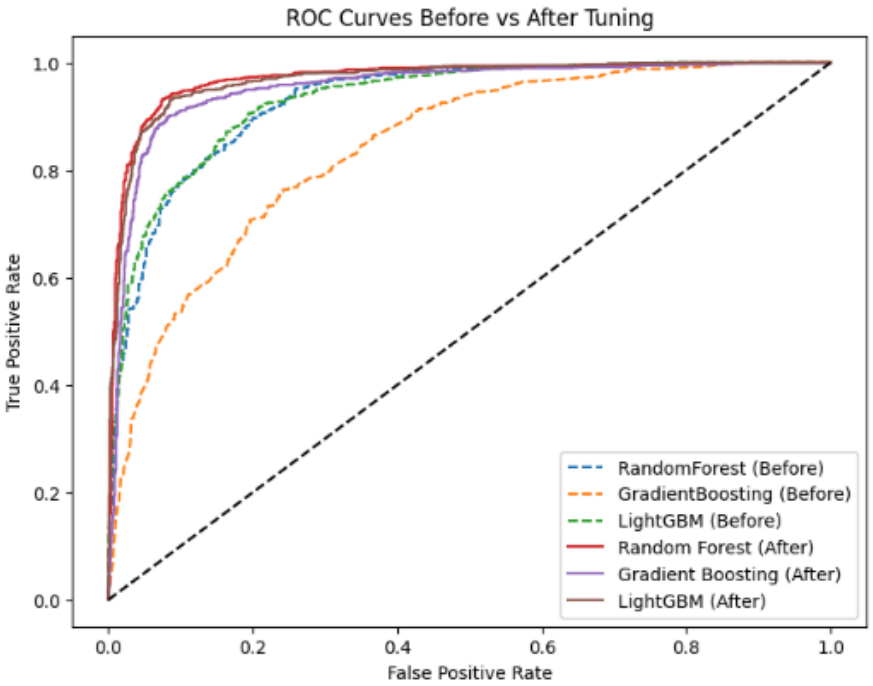
LightGBM, which already performed well at baseline, also improved notably across all metrics. It's also reflecting the effectiveness of the tuned parameters (e.g., n_estimators=200, learning_rate=0.1, max_depth=5, num_leaves=31).

Overall Comparison and Best Model:

All three models benefited significantly from hyperparameter tuning:

- Random Forest emerges as the top performer, achieving the highest test accuracy (0.9274) and ROC AUC (0.9714), indicating it is the most robust and discriminative model for this dataset.
- LightGBM closely follows Random Forest, demonstrating strong predictive capability and efficiency.
- Gradient Boosting, despite its lower baseline, achieved impressive performance after tuning, highlighting its potential when optimized.

These results confirm that careful hyperparameter tuning is crucial for maximizing model performance, and the models are now well-configured for accurately predicting stock target movements based on the selected features.



11.Model Interpretability (SHAP Analysis)

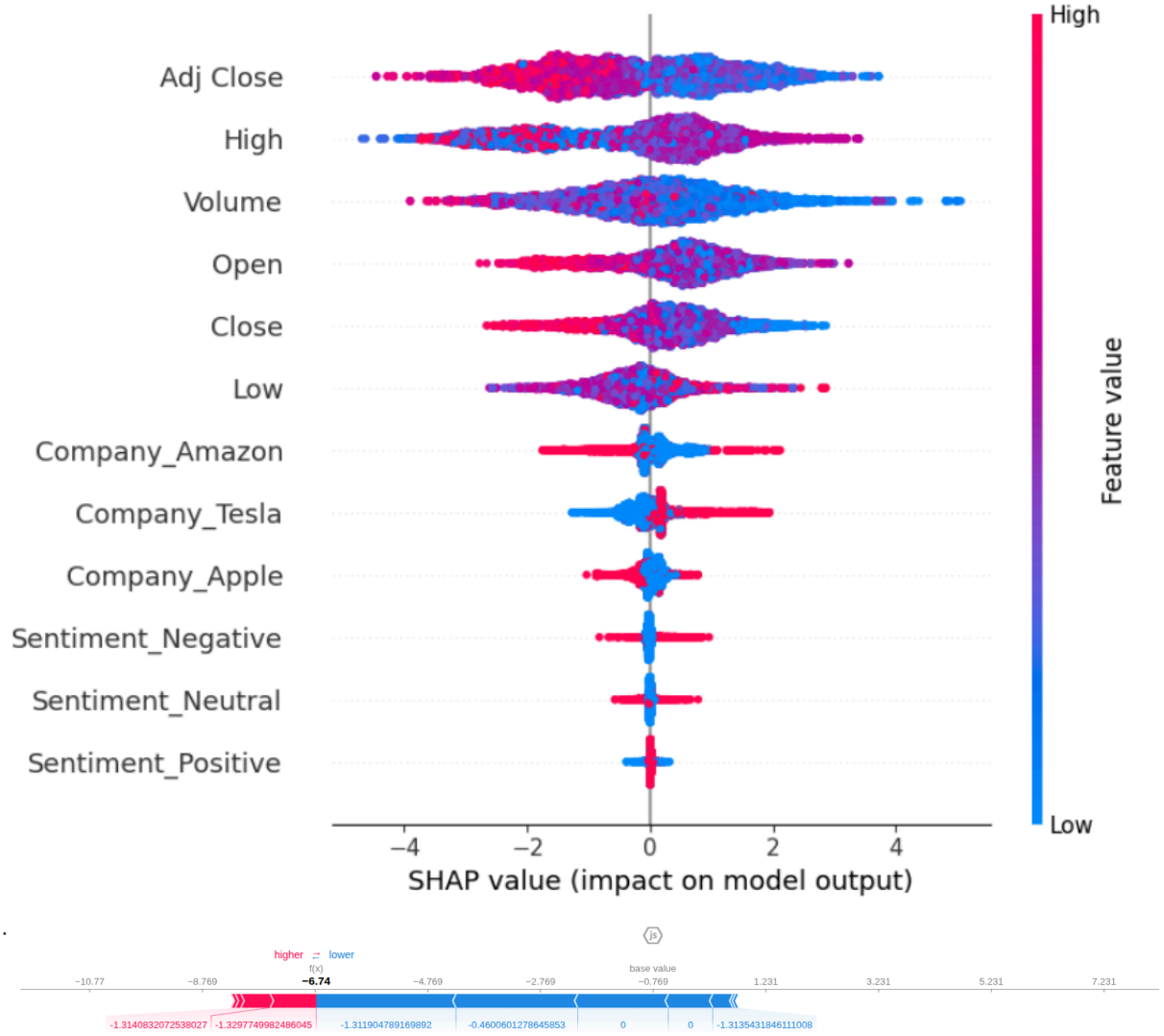


Fig XV: SHAP analysis

SHAP analysis was employed to interpret both global and local behavior of the final tuned tree-based models. The SHAP summary plot revealed that price-based features (Adj Close, Open, High, Low, Close) and Volume were the dominant contributors to prediction outcomes, consistent with statistical significance identified via Mann-Whitney U tests. Company identity exhibited weak but non-zero influence, aligning with Chi-square results indicating a significant yet small effect size. Sentiment features showed minimal individual contribution, confirming earlier statistical findings.However, their non-zero SHAP values suggest marginal utility when interacting with price features. The SHAP force plot further demonstrated how individual predictions are formed by additive feature contributions, providing transparent, instance-level explanations of model decisions. Overall, SHAP analysis validates the feature selection strategy and confirms that the model relies on economically meaningful signals rather than spurious correlations.

12.Real life prediction

Figure XVI presents a SHAP-based local explanation for a one-day-ahead stock movement prediction. Although company affiliation (Amazon) and moderate trading volume exert upward pressure on the prediction, these effects are outweighed by adverse price structure signals, including a lower adjusted close, weaker intraday minimum price, and unfavorable sentiment score. Notably, while sentiment features were not statistically significant at the dataset level, SHAP analysis reveals their capacity to influence individual predictions in conjunction with price variables. The resulting low prediction confidence reflects the presence of conflicting market signals, demonstrating appropriate model uncertainty and reinforcing the robustness of the explainability framework.

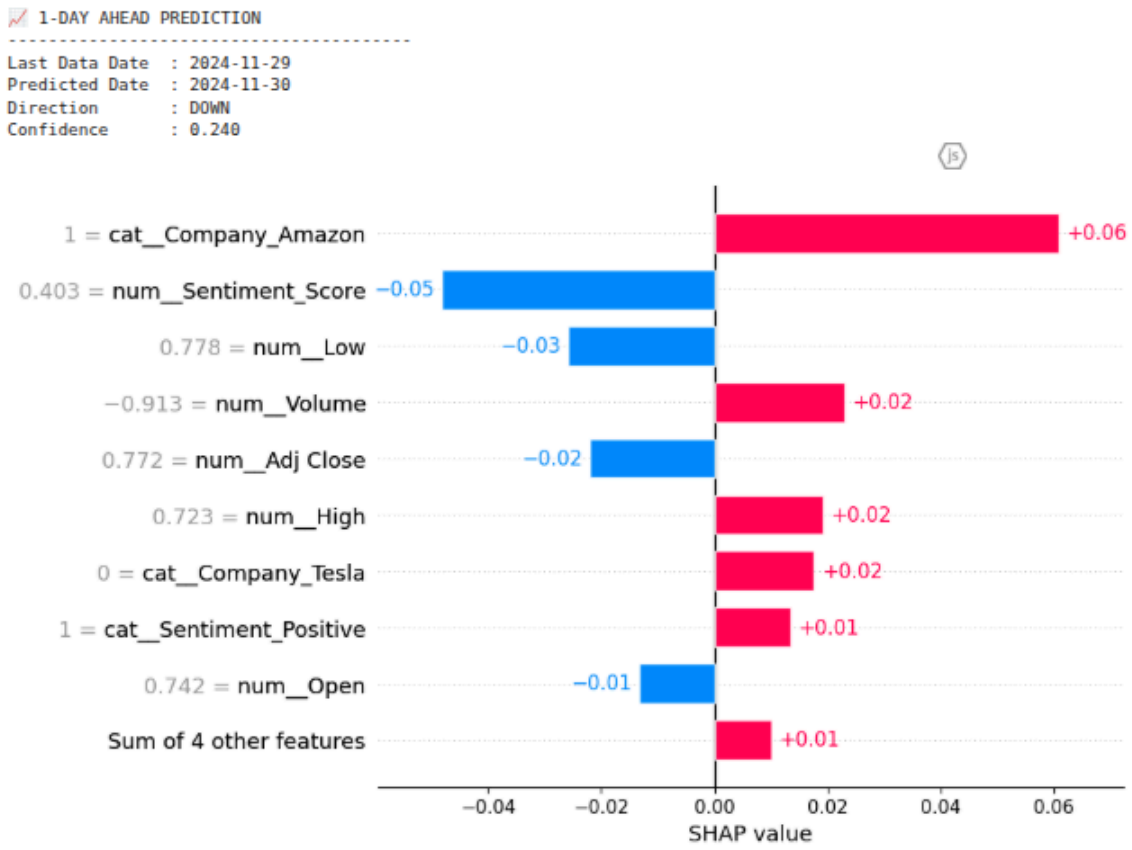


Fig XVI: SHAP analysis on real life data

Reference list

Agrawal, M. and Mukherjee, A. (2025). Predicting Stock Market Trends Using Machine Learning and Sentiment Analysis. SoutheastCon 2025, [online] pp.1001–1006. doi:https://doi.org/10.1109/southeastcon56624.2025.10971605.

Bouktif, S., Fiaz, A. and Awad, M. (2020). Augmented Textual Features-Based Stock Market Prediction. IEEE Access, 8, pp.40269–40282. doi:https://doi.org/10.1109/access.2020.2976725.

Koukaras, P., Nousi, C. and Tjortjis, C. (2022). Stock Market Prediction Using Microblogging Sentiment Analysis and Machine Learning. Telecom, 3(2), pp.358–378. doi:https://doi.org/10.3390/telecom3020019.
\
Mankar, T., Hotchandani, T., Madhwani, M., Chidrawar, A. and Lifna, C.S. (2018). Stock Market Prediction based on Social Sentiments using Machine Learning. 2018 International Conference on Smart City and Emerging Technology (ICSCET). doi:https://doi.org/10.1109/icscet.2018.8537242.

Mwamba, S. (2020). Sentiment, emotions and stock market predictability in developed and emerging markets. [online] GLO Discussion Paper Series. Available at: https://ideas.repec.org/p/zbw/glodps/502.html.

Nousi, C. and Tjortjis, C. (2021). A Methodology for Stock Movement Prediction Using Sentiment Analysis on Twitter and StockTwits Data. [online] IEEE Xplore. doi:https://doi.org/10.1109/SEEDA-CECNSM53056.2021.9566242.

Pagolu, V.S., Reddy, K.N., Panda, G. and Majhi, B. (2016). Sentiment analysis of Twitter data for predicting stock market movements. [online] IEEE Xplore. doi:https://doi.org/10.1109/SCOPES.2016.7955659.

Reddy, S. and Kumar, A. (2021). PREDICTION OF STOCK WORTH MOVEMENT USING MACHINE LEARNING K Geetha 1. [online] International Research Journal of Engineering and Technology. Available at: https://www.irjet.net/archives/V8/i5/IRJET-V8I5369.pdf [Accessed 25 Jan. 2026].